. Supervised ML for Intrusion Detection 4 marks Using the NSL-KDD or CICIDS-2017 dataset, build and evaluate supervised ML classifiers for network intrusion detection.

Tasks to be completed:

• Load and explore the dataset; describe features and class distribution.

• Apply at least THREE supervised classifiers: Decision Tree, Random Forest, and Support Vector Machine (SVM).

• Split data into 80% training and 20% testing sets. Apply label encoding where required.

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report,
    ConfusionMatrixDisplay
)

url =
"https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain
%2B.txt"

columns = [
    "duration", "protocol_type", "service", "flag", "src_bytes",
    "dst_bytes", "land", "wrong_fragment", "urgent", "hot",
    "num_failed_logins", "logged_in", "num_compromised",
    "root_shell", "su_attempted", "num_root", "num_file_creations",
    "num_shells", "num_access_files", "num_outbound_cmds",
    "is_host_login", "is_guest_login", "count", "srv_count",
    "serror_rate", "srv_serror_rate", "rerror_rate",
    "srv_rerror_rate", "same_srv_rate", "diff_srv_rate",
    "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate",
    "dst_host_same_src_port_rate", "dst_host_srv_diff_host_rate",
    "dst_host_serror_rate", "dst_host_srv_serror_rate",
    "dst_host_rerror_rate", "dst_host_srv_rerror_rate",
    "label", "difficulty"
]

df = pd.read_csv(url, header=None, names=columns)
```

```
print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)
print("First five rows:")
display(df.head())

print("Dataset information:")
df.info()

print("\nMissing values:", df.isnull().sum().sum())

print("\nClass distribution:")
print(df["label"].value_counts())

df["label"].value_counts().plot(kind="bar", figsize=(10, 4))
plt.title("Distribution of Network Traffic Classes")
plt.xlabel("Traffic Class")
plt.ylabel("Number of Records")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

```
Dataset loaded successfully!
Dataset shape: (125973, 43)
First five rows:
```

{"type":"dataframe"}

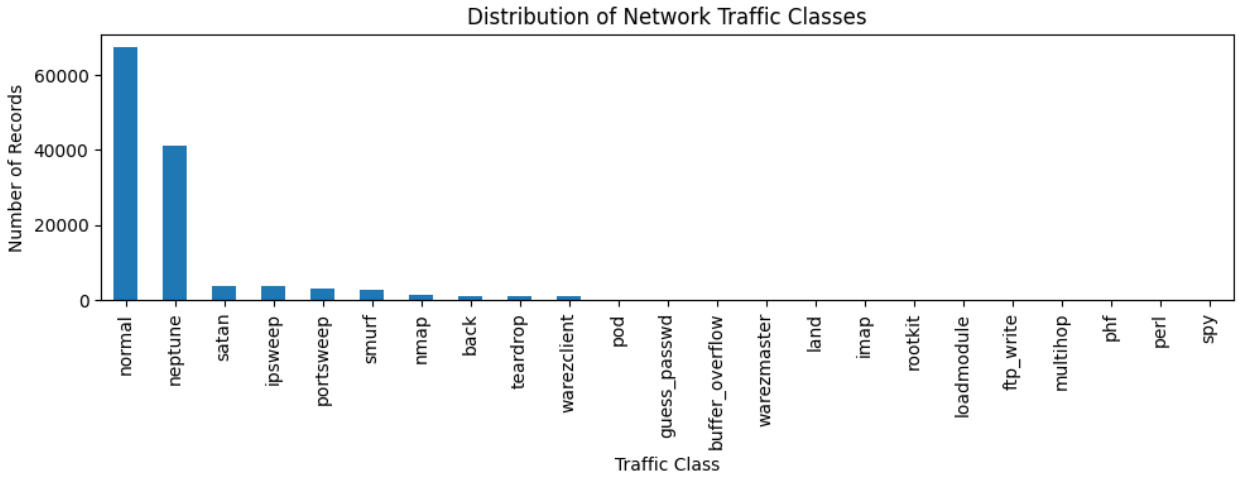
```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 125973 entries, 0 to 125972
Data columns (total 43 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   duration                     125973 non-null  int64
 1   protocol_type                125973 non-null  object
 2   service                      125973 non-null  object
 3   flag                         125973 non-null  object
 4   src_bytes                    125973 non-null  int64
 5   dst_bytes                    125973 non-null  int64
 6   land                         125973 non-null  int64
 7   wrong_fragment               125973 non-null  int64
 8   urgent                       125973 non-null  int64
 9   hot                          125973 non-null  int64
 10  num_failed_logins            125973 non-null  int64
 11  logged_in                    125973 non-null  int64
 12  num_compromised              125973 non-null  int64
 13  root_shell                   125973 non-null  int64
 14  su_attempted                 125973 non-null  int64
 15  num_root                     125973 non-null  int64
```

```
 16  num_file_creations           125973 non-null  int64
 17  num_shells                   125973 non-null  int64
 18  num_access_files             125973 non-null  int64
 19  num_outbound_cmds            125973 non-null  int64
 20  is_host_login                125973 non-null  int64
 21  is_guest_login               125973 non-null  int64
 22  count                        125973 non-null  int64
 23  srv_count                    125973 non-null  int64
 24  serror_rate                  125973 non-null  float64
 25  srv_serror_rate              125973 non-null  float64
 26  rerror_rate                  125973 non-null  float64
 27  srv_rerror_rate              125973 non-null  float64
 28  same_srv_rate                125973 non-null  float64
 29  diff_srv_rate                125973 non-null  float64
 30  srv_diff_host_rate           125973 non-null  float64
 31  dst_host_count               125973 non-null  int64
 32  dst_host_srv_count           125973 non-null  int64
 33  dst_host_same_srv_rate       125973 non-null  float64
 34  dst_host_diff_srv_rate       125973 non-null  float64
 35  dst_host_same_src_port_rate  125973 non-null  float64
 36  dst_host_srv_diff_host_rate  125973 non-null  float64
 37  dst_host_serror_rate         125973 non-null  float64
 38  dst_host_srv_serror_rate     125973 non-null  float64
 39  dst_host_rerror_rate         125973 non-null  float64
 40  dst_host_srv_rerror_rate     125973 non-null  float64
 41  label                        125973 non-null  object
 42  difficulty                   125973 non-null  int64
dtypes: float64(15), int64(24), object(4)
memory usage: 41.3+ MB

Missing values: 0

Class distribution:
label
normal             67343
neptune            41214
satan               3633
ipsweep             3599
portsweep           2931
smurf               2646
nmap                1493
back                 956
teardrop             892
warezclient          890
pod                  201
guess_passwd          53
buffer_overflow       30
warezmaster           20
land                  18
```

```
imap                    11
rootkit                 10
loadmodule               9
ftp_write                8
multihop                 7
phf                      4
perl                     3
spy                      2
Name: count, dtype: int64
```




```python
# 0 = Normal traffic, 1 = Attack traffic
y = (df["label"] != "normal").astype(int)

# Remove target and difficulty columns
X = df.drop(columns=["label", "difficulty"])

# Encode categorical features
X = pd.get_dummies(
    X,
    columns=["protocol_type", "service", "flag"],
    dtype=int
)

print("Feature shape:", X.shape)
print("\nClass counts:")
print(y.value_counts().rename({0: "Normal", 1: "Attack"}))
```

```
Feature shape: (125973, 122)

Class counts:
label
Normal    67343
```

```
Attack    58630
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training records:", X_train.shape[0])
print("Testing records:", X_test.shape[0])
```

```
Training records: 100778
Testing records: 25195
```

```
dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)

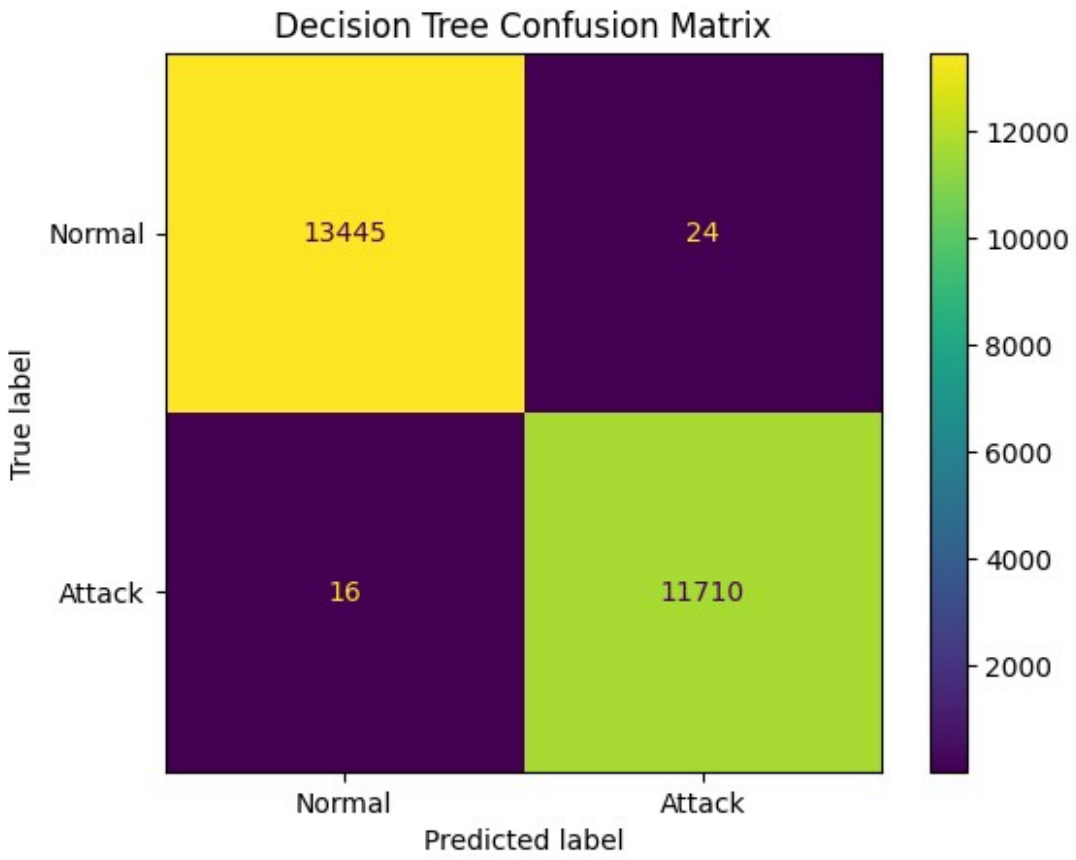
print("Decision Tree Results")
print(classification_report(
    y_test, dt_pred,
    target_names=["Normal", "Attack"],
    zero_division=0
))

ConfusionMatrixDisplay.from_predictions(
    y_test, dt_pred,
    display_labels=["Normal", "Attack"]
)
plt.title("Decision Tree Confusion Matrix")
plt.show()
```

```
Decision Tree Results
              precision    recall  f1-score   support

      Normal       1.00      1.00      1.00     13469
      Attack       1.00      1.00      1.00     11726

    accuracy                           1.00     25195
   macro avg       1.00      1.00      1.00     25195
weighted avg       1.00      1.00      1.00     25195
```




```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

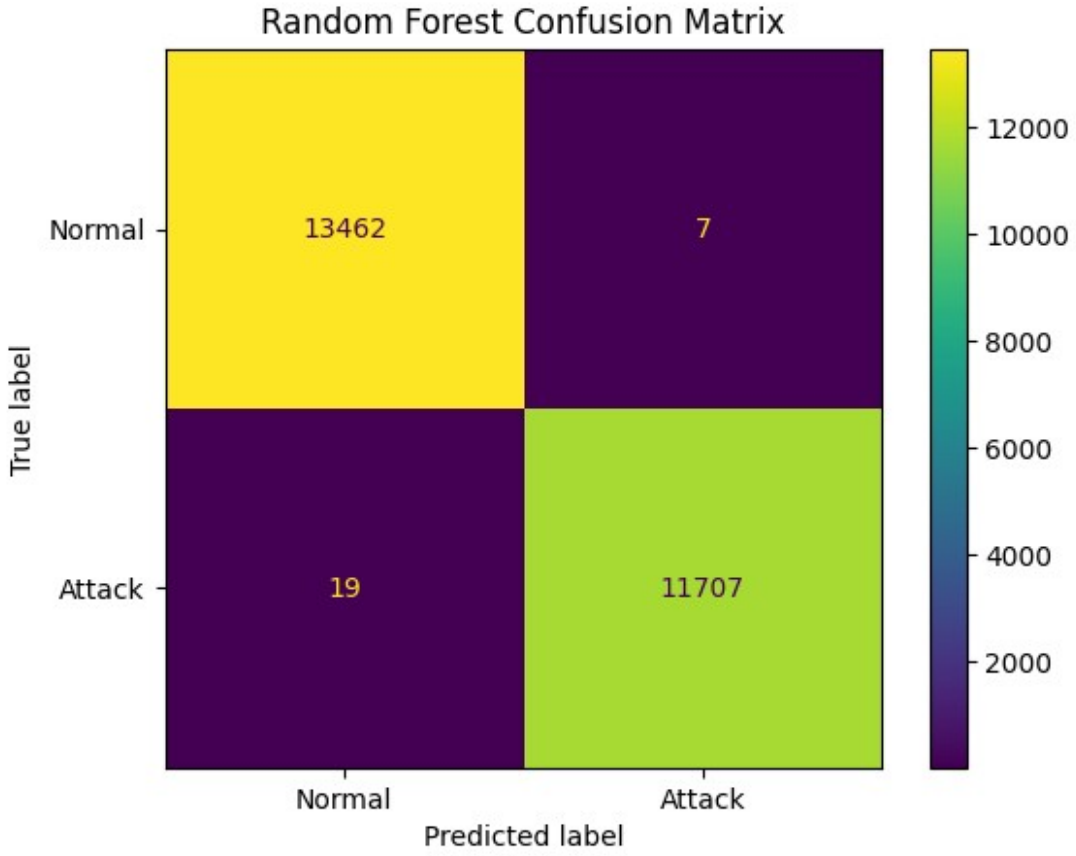
print("Random Forest Results")
print(classification_report(
    y_test, rf_pred,
    target_names=["Normal", "Attack"],
    zero_division=0
))

ConfusionMatrixDisplay.from_predictions(
    y_test, rf_pred,
    display_labels=["Normal", "Attack"]
)
plt.title("Random Forest Confusion Matrix")
plt.show()
```

```
Random Forest Results
              precision    recall  f1-score   support

      Normal       1.00      1.00      1.00     13469
      Attack       1.00      1.00      1.00     11726

    accuracy                           1.00     25195
   macro avg       1.00      1.00      1.00     25195
weighted avg       1.00      1.00      1.00     25195
```




```python
svm_model = make_pipeline(
    StandardScaler(),
    LinearSVC(
        max_iter=5000,
        dual=False,
        random_state=42
    )
)

svm_model.fit(X_train, y_train)
svm_pred = svm_model.predict(X_test)
```

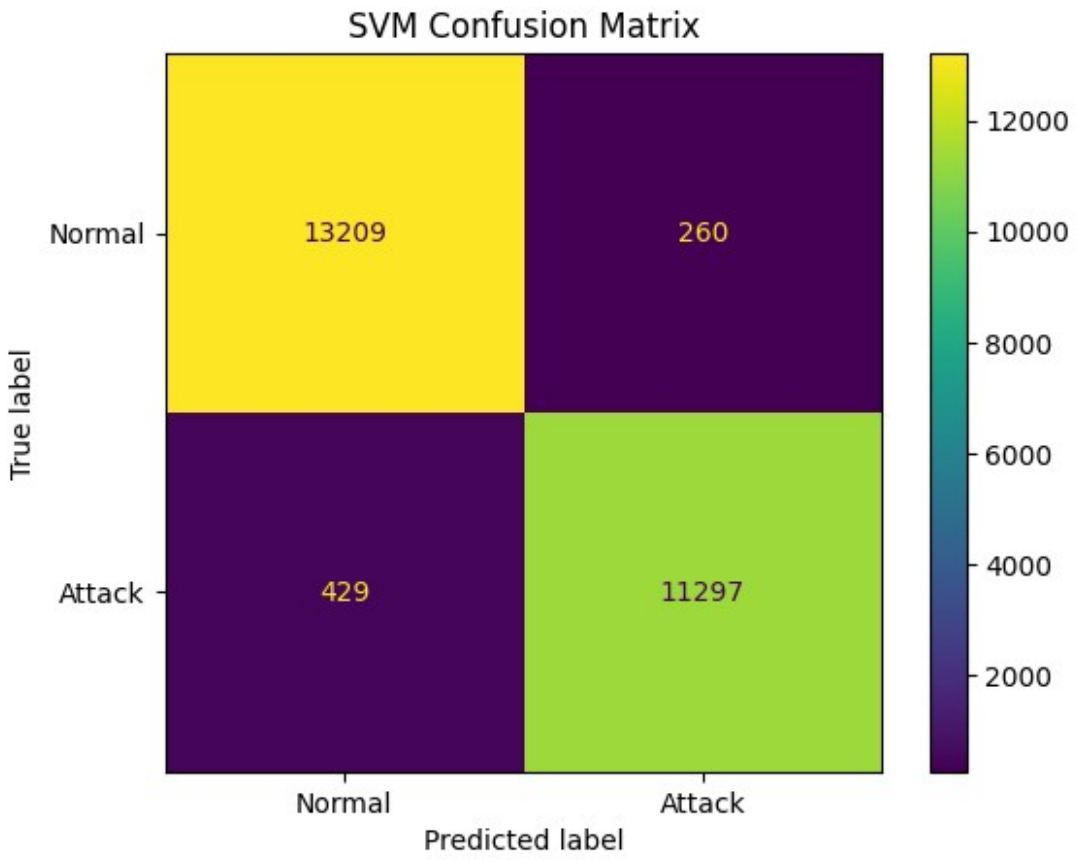
```python
print("SVM Results")
print(classification_report(
    y_test, svm_pred,
    target_names=["Normal", "Attack"],
    zero_division=0
))

ConfusionMatrixDisplay.from_predictions(
    y_test, svm_pred,
    display_labels=["Normal", "Attack"]
)
plt.title("SVM Confusion Matrix")
plt.show()
```

```
SVM Results
              precision    recall  f1-score   support

      Normal       0.97      0.98      0.97     13469
      Attack       0.98      0.96      0.97     11726

    accuracy                           0.97     25195
   macro avg       0.97      0.97      0.97     25195
weighted avg       0.97      0.97      0.97     25195
```




```
models_predictions = {
    "Decision Tree": dt_pred,
    "Random Forest": rf_pred,
    "SVM": svm_pred
}

results = []

for name, pred in models_predictions.items():
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(
            y_test, pred, zero_division=0
        ),
        "Recall": recall_score(
            y_test, pred, zero_division=0
        ),
        "F1 Score": f1_score(
            y_test, pred, zero_division=0
        )
    })
```

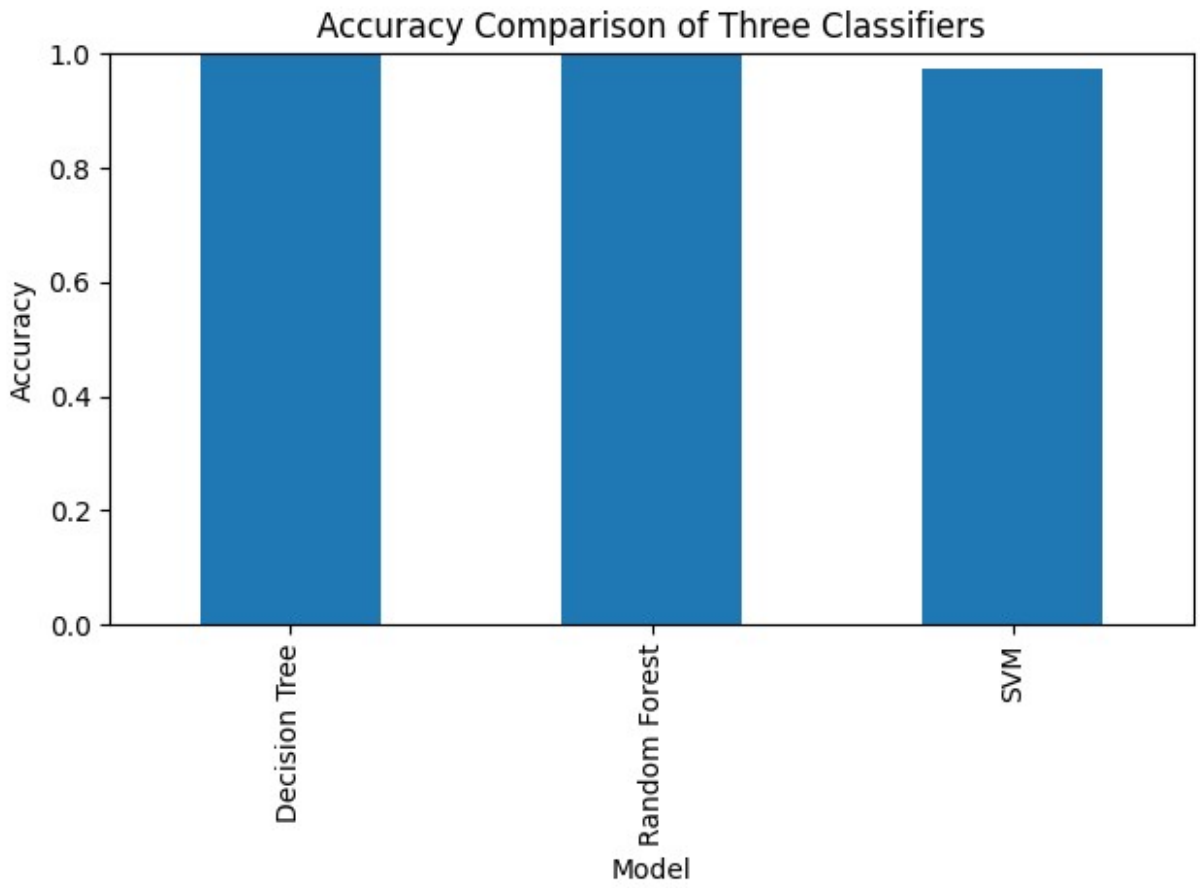
```
results_df = pd.DataFrame(results)

print("Final Model Comparison:")
display(results_df.round(4))

results_df.set_index("Model")["Accuracy"].plot(
    kind="bar", ylim=(0, 1),
    title="Accuracy Comparison of Three Classifiers"
)
plt.ylabel("Accuracy")
plt.tight_layout()
plt.show()
```

```
Final Model Comparison:
```


```
{"summary":"{\n  \"name\": \"plt\",\n  \"rows\": 3,\n  \"fields\": [\n
{\n      \"column\": \"Model\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 3,\n
\"samples\": [\n          \"Decision Tree\",\n          \"Random
Forest\",\n          \"SVM\"\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Accuracy\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.015014104479899319,\n        \"min\":
0.9727,\n        \"max\": 0.999,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.9984,\n          0.999,\n          0.9727\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Precision\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.01225982599115226,\n        \"min\":
0.9775,\n        \"max\": 0.9994,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.998,\n          0.9994,\n          0.9775\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Recall\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.02026524117793813,\n        \"min\": 0.9634,\n
\"max\": 0.9986,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.9986,\n          0.9984,\n
0.9634\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"F1
Score\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.016284041267449523,\n        \"min\": 0.9704,\n
\"max\": 0.9989,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.9983,\n          0.9989,\n
0.9704\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```



Apply unsupervised Machine Learning techniques to detect anomalies in network traffic or system log data.

Tasks to be completed:

• Select an appropriate dataset (e.g., KDD Cup 1999, UNSW-NB15, or a provided log dataset).

• Apply K-Means Clustering: determine optimal k using the Elbow Method.

• Apply DBSCAN: justify parameter selection (eps, min_samples).

• Apply Isolation Forest for outlier/anomaly detection.

• Visualise clusters using PCA or t-SNE dimensionality reduction.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.ensemble import IsolationForest
```

```python
from sklearn.decomposition import PCA

df = pd.read_csv(

"https://raw.githubusercontent.com/baonq-me/kdd-cup-1999/master/kddcup
.data_10_percent.gz",
    header=None,
    compression="gzip"
)

X = df.iloc[:, :-1].copy()
y = df.iloc[:, -1].copy()

print("Dataset shape:", df.shape)
display(df.head())
print("Labels:")
print(y.value_counts().head())
```

```
Dataset shape: (494021, 42)
```

{"type":"dataframe"}

```
Labels:
41
smurf.      280790
neptune.    107201
normal.      97278
back.         2203
satan.        1589
Name: count, dtype: int64
```

```python
# Part 2: Preprocess and scale the data

# Make sure X is a DataFrame
X = pd.DataFrame(X).copy()

# Convert every column to numeric
for col in X.columns:
    X[col] = pd.to_numeric(X[col], errors="coerce")

# Replace invalid values
X = X.replace([np.inf, -np.inf], np.nan)
X = X.fillna(0)

# Sample data for faster processing
X = X.sample(n=min(5000, len(X)), random_state=42)

# Scale the features
try:
    scaler = StandardScaler()
```

```
    X_scaled = scaler.fit_transform(X)

    print("Preprocessing completed successfully!")
    print("X_scaled shape:", X_scaled.shape)

except Exception as e:
    print("Scaling error:", repr(e))
    print("X shape:", X.shape)
    print("Data types:\n", X.dtypes)
```

```
Scaling error: TypeError("Feature names are only supported if all
input features have string names, but your input has ['int', 'str'] as
feature name / column name types. If you want feature names to be
stored and validated, you must convert them all to strings, by using
X.columns = X.columns.astype(str) for example. Otherwise you can
remove feature / column names from your input data, or convert them
all to a non-string data type.")
X shape: (2000, 118)
Data types:
 0       int64
4       int64
5       int64
6       int64
7       int64
        ...
3_S1    int64
3_S2    int64
3_S3    int64
3_SF    int64
3_SH    int64
Length: 118, dtype: object
```

```
# PART 3: ELBOW METHOD AND K-MEANS CLUSTERING

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Step 1: Check dataset
if "X" not in globals():
    raise ValueError("Run Part 1 first to load the dataset.")

# Step 2: Prepare features
X_part3 = pd.DataFrame(X).copy()

# Make all column names strings
```

```
X_part3.columns = X_part3.columns.astype(str)

# Convert values to numeric
X_part3 = X_part3.apply(pd.to_numeric, errors="coerce")

# Handle missing and infinite values
X_part3 = X_part3.replace([np.inf, -np.inf], np.nan)
X_part3 = X_part3.fillna(0)

# Remove columns with no variation
X_part3 = X_part3.loc[:, X_part3.nunique() > 1]

# Sample up to 5000 rows for faster clustering
X_part3 = X_part3.sample(
    n=min(5000, len(X_part3)),
    random_state=42
)

if X_part3.empty or X_part3.shape[1] == 0:
    raise ValueError("No usable feature columns found.")

print("Prepared data shape:", X_part3.shape)

# Step 3: Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_part3)

print("Scaling completed successfully!")
print("X_scaled shape:", X_scaled.shape)

# Step 4: Elbow Method
inertia = []

for k in range(1, 8):
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    model.fit(X_scaled)
    inertia.append(model.inertia_)

# Step 5: Plot Elbow graph
plt.figure(figsize=(8, 5))
plt.plot(range(1, 8), inertia, marker="o")
plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.xticks(range(1, 8))
plt.grid(True)
```

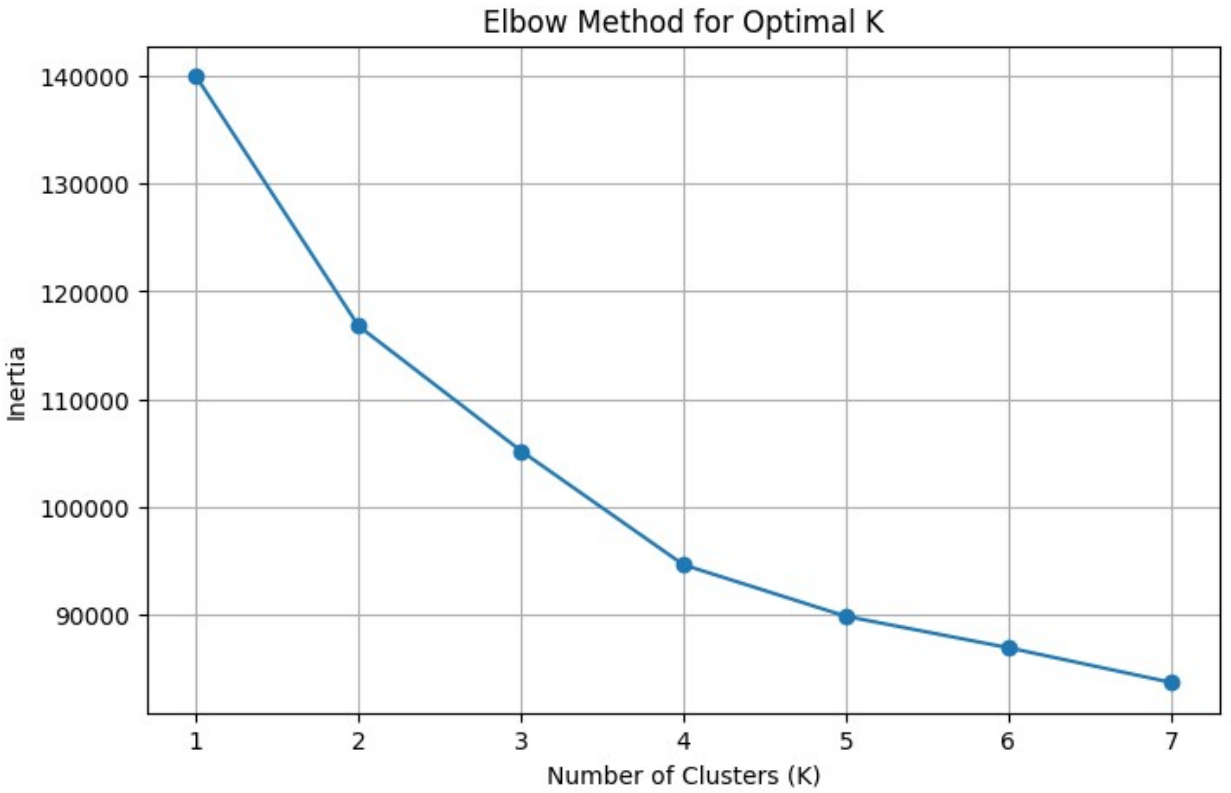
```
plt.show()

# Step 6: Apply K-Means
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

kmeans_labels = kmeans.fit_predict(X_scaled)

print("K-Means clustering completed successfully!")
print("\nNumber of points in each cluster:")
print(
    pd.Series(kmeans_labels)
    .value_counts()
    .sort_index()
)
```

```
Prepared data shape: (2000, 70)
Scaling completed successfully!
X_scaled shape: (2000, 70)
```



```
K-Means clustering completed successfully!

Number of points in each cluster:
0    1178
1     384
2     438
Name: count, dtype: int64
```

```
# DBSCAN groups dense points and identifies noise
dbscan = DBSCAN(
    eps=3.0,
    min_samples=10
)

dbscan_labels = dbscan.fit_predict(X_scaled)

n_clusters = len(set(dbscan_labels) - {-1})
n_noise = np.sum(dbscan_labels == -1)

print("DBSCAN completed.")
print("Number of clusters:", n_clusters)
print("Potential anomaly/noise points:", n_noise)
print("Cluster labels:", np.unique(dbscan_labels))
```

```
DBSCAN completed.
Number of clusters: 10
Potential anomaly/noise points: 116
Cluster labels: [-1  0  1  2  3  4  5  6  7  8  9]
```

```
# Detect unusual network connections
iso_forest = IsolationForest(
    contamination=0.05,
    random_state=42
)

iso_labels = iso_forest.fit_predict(X_scaled)

# -1 = anomaly, 1 = normal
anomalies = iso_labels == -1

print("Isolation Forest completed.")
print("Normal records:", np.sum(iso_labels == 1))
print("Potential anomalies:", np.sum(anomalies))
```

```
Isolation Forest completed.
Normal records: 1900
Potential anomalies: 100
```

```python
# Reduce features to two dimensions for visualisation
X_pca = PCA(n_components=2).fit_transform(X_scaled)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=kmeans_labels, s=10
)
axes[0].set_title("K-Means Clusters")

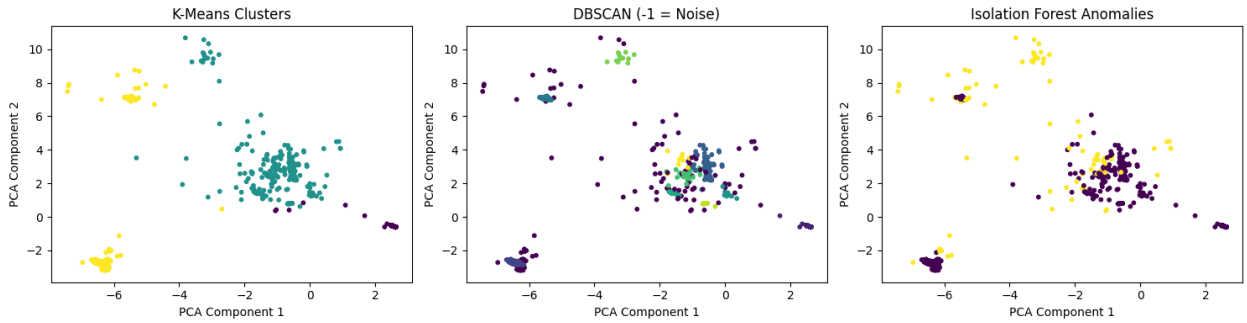
axes[1].scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=dbscan_labels, s=10
)
axes[1].set_title("DBSCAN (-1 = Noise)")

axes[2].scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=anomalies, s=10
)
axes[2].set_title("Isolation Forest Anomalies")

for ax in axes:
    ax.set_xlabel("PCA Component 1")
    ax.set_ylabel("PCA Component 2")

plt.tight_layout()
plt.show()

print("\nFinal Summary")
print("K-Means clusters:", len(np.unique(kmeans_labels)))
print("DBSCAN noise points:", n_noise)
print("Isolation Forest anomalies:", np.sum(anomalies))
```




```
Final Summary
K-Means clusters: 3
```

```
DBSCAN noise points: 116
Isolation Forest anomalies: 100
```

Prepare a structured comparative case study examining how three different ML techniques have been applied to the same cyber security problem domain.

Suggested problem domains (choose one with faculty approval):

• Spam and phishing email detection

• Malware classification and family identification

• Credit card fraud detection

• Network intrusion detection

For each technique, document: algorithm description, dataset used, performance metrics reported in literature, advantages, and limitations.

```
# QUESTION 3: SPAM EMAIL DETECTION
# Part 1: Load Dataset

%pip install -q ucimlrepo

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from ucimlrepo import fetch_ucirepo

# Load UCI Spambase dataset
spambase = fetch_ucirepo(id=94)

X = spambase.data.features.copy()
y = spambase.data.targets.iloc[:, 0].copy()

# Clean column names
X.columns = X.columns.astype(str)

# Convert target to numeric
y = pd.to_numeric(y, errors="coerce")

# Remove rows with missing target values
valid_rows = y.notna()
X = X.loc[valid_rows].reset_index(drop=True)
y = y.loc[valid_rows].astype(int).reset_index(drop=True)

print("Dataset loaded successfully!")
print("Dataset shape:", X.shape)
print("Target shape:", y.shape)
```

```python
print("\nFirst five rows:")
display(X.head())

print("\nSpam distribution (0 = Not Spam, 1 = Spam):")
print(y.value_counts().sort_index())
```

```
Dataset loaded successfully!
Dataset shape: (4601, 57)
Target shape: (4601,)

First five rows:
```

{"type":"dataframe"}

```
Spam distribution (0 = Not Spam, 1 = Spam):
Class
0    2788
1    1813
Name: count, dtype: int64
```

```python
# Part 2: Data Preprocessing

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Convert feature values to numeric
X = X.apply(pd.to_numeric, errors="coerce")
X = X.replace([np.inf, -np.inf], np.nan)
X = X.fillna(X.median())

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Scale features for models that benefit from scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Preprocessing completed!")
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("Number of features:", X_train.shape[1])
```

```
Preprocessing completed!
Training samples: 3680
Testing samples: 921
Number of features: 57


# Part 3: Train and Evaluate Models

from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# Define models
models = {
    "Naive Bayes": GaussianNB(),
    "Logistic Regression": LogisticRegression(
        max_iter=1000,
        random_state=42
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        n_jobs=-1
    )
}

results = []
predictions = {}

for name, model in models.items():

    # Use scaled features for Naive Bayes and Logistic Regression
    if name in ["Naive Bayes", "Logistic Regression"]:
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    predictions[name] = y_pred
```

```
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test, y_pred, zero_division=0
        ),
        "Recall": recall_score(
            y_test, y_pred, zero_division=0
        ),
        "F1 Score": f1_score(
            y_test, y_pred, zero_division=0
        )
    })

    print("\n", "=" * 45)
    print("Model:", name)
    print(classification_report(
        y_test,
        y_pred,
        target_names=["Not Spam", "Spam"],
        zero_division=0
    ))

results_df = pd.DataFrame(results)

print("\nModel Comparison:")
display(results_df.round(4))
```

```
 =============================================
Model: Naive Bayes
              precision    recall  f1-score   support

    Not Spam       0.97      0.75      0.84       558
        Spam       0.71      0.96      0.82       363

    accuracy                           0.83       921
   macro avg       0.84      0.85      0.83       921
weighted avg       0.87      0.83      0.83       921


 =============================================
Model: Logistic Regression
              precision    recall  f1-score   support

    Not Spam       0.93      0.95      0.94       558
        Spam       0.92      0.90      0.91       363

    accuracy                           0.93       921
   macro avg       0.93      0.92      0.93       921
```

```
weighted avg       0.93      0.93      0.93       921


 ============================================
Model: Random Forest
              precision    recall  f1-score   support

    Not Spam       0.94      0.97      0.95       558
        Spam       0.95      0.91      0.93       363

    accuracy                           0.94       921
   macro avg       0.95      0.94      0.94       921
weighted avg       0.94      0.94      0.94       921


Model Comparison:
```


```
{"summary":"{\n  \"name\": \"display(results_df\",\n  \"rows\": 3,\n
\"fields\": [\n    {\n      \"column\": \"Model\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n          \"Naive
Bayes\",\n          \"Logistic Regression\",\n          \"Random
Forest\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Accuracy\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.06063805185964778,\n        \"min\":
0.8328,\n        \"max\": 0.9446,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.8328,\n          0.9294,\n
0.9446\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Precision\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.12775376054478138,\n        \"min\":
0.7146,\n        \"max\": 0.9483,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.7146,\n          0.9209,\n
0.9483\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Recall\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.032283948539999455,\n        \"min\": 0.8981,\n
\"max\": 0.9587,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.9587,\n          0.8981,\n
0.9091\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"F1
Score\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.058511394901620106,\n        \"min\": 0.8188,\n
\"max\": 0.9283,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.8188,\n          0.9093,\n
0.9283\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```

```python
# Part 4: Visualization and Comparison

from sklearn.metrics import ConfusionMatrixDisplay

# Compare evaluation metrics
metrics = ["Accuracy", "Precision", "Recall", "F1 Score"]

results_df.set_index("Model")[metrics].plot(
    kind="bar",
    figsize=(11, 6),
    ylim=(0, 1),
    rot=0
)

plt.title("Spam Detection: Model Performance Comparison")
plt.ylabel("Score")
plt.xlabel("Machine Learning Model")
plt.legend(loc="lower right")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

# Display confusion matrix for each model
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, y_pred) in zip(axes, predictions.items()):
    cm = confusion_matrix(y_test, y_pred, labels=[0, 1])

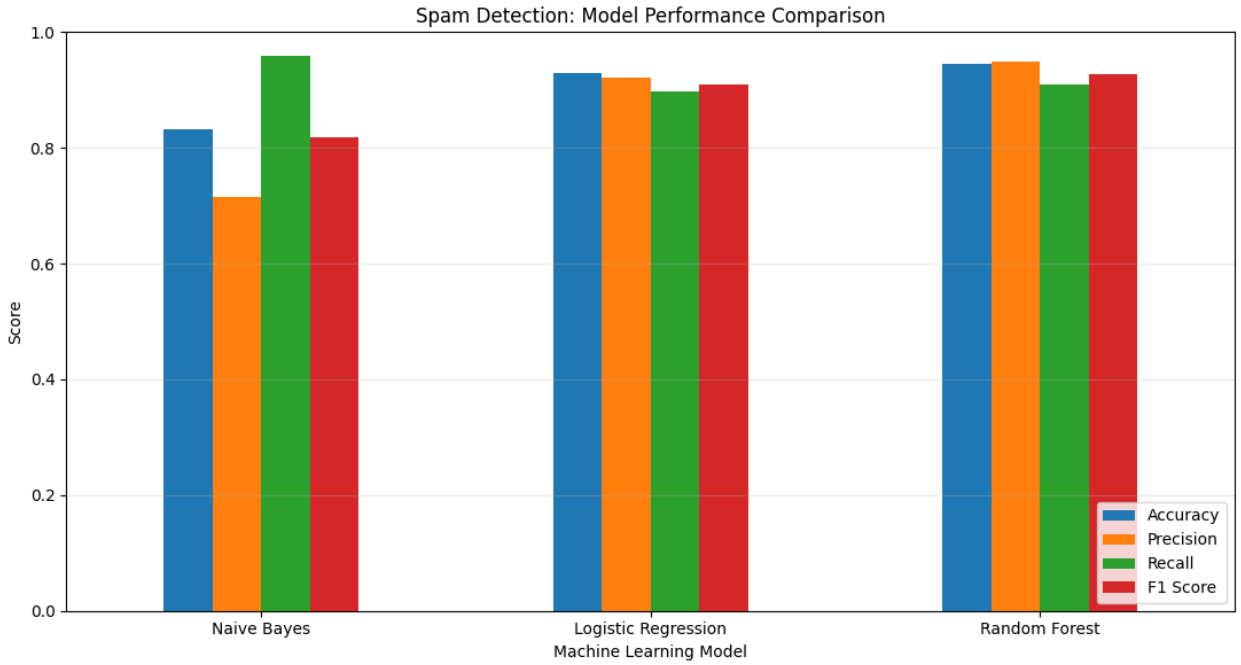
    display_obj = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["Not Spam", "Spam"]
    )

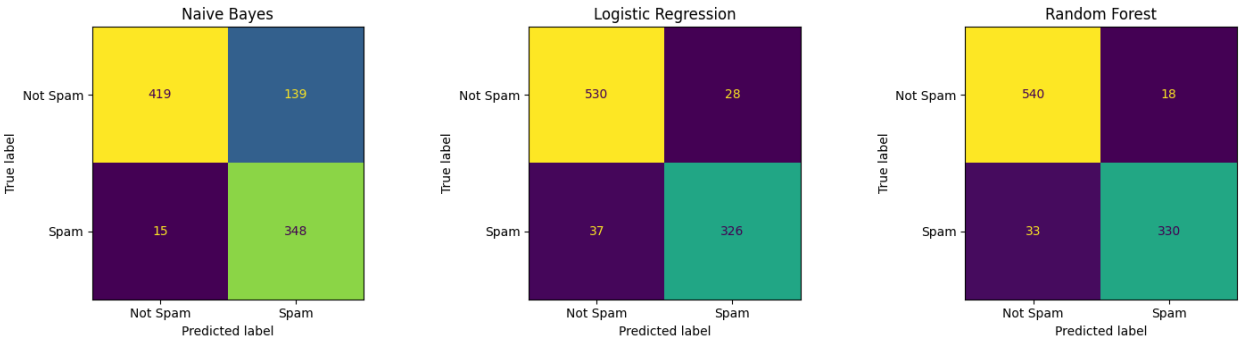
    display_obj.plot(ax=ax, colorbar=False)
    ax.set_title(name)

plt.tight_layout()
plt.show()

# Identify the model with the highest F1 score
best_model = results_df.loc[
    results_df["F1 Score"].idxmax()
]

print("Best model based on F1 Score:", best_model["Model"])
print("F1 Score:", round(best_model["F1 Score"], 4))
print("Accuracy:", round(best_model["Accuracy"], 4))
```






```
Best model based on F1 Score: Random Forest
F1 Score: 0.9283
Accuracy: 0.9446
```